REFLEXIÓN

1. El cambio: Proyecto 1 vs Proyecto Final La diferencia fundamental al pasar de programación estructurada a POO fue el principio de Responsabilidad Única. En el Proyecto 1, funciones como `registrar_venta()` hacían de todo: pedían inputs, restaban números, evaluaban reglas de negocio, abrían archivos CSV y generaban registros en bitácoras. Era un código difícil de entender. Al aplicar POO, el sistema se volvió modular. Noté ventajas inmediatas en la seguridad de los datos y en el rendimiento: al cargar los datos en un diccionario dentro de la clase `Inventario` en lugar de leer el disco duro en cada iteración de un ciclo `for`, las búsquedas pasaron de ser lentas a ser instantáneas.

2. Decisión de clases y responsabilidades Decidí separar el sistema en entidades lógicas para aislar la interfaz de la base de datos. `Producto` y `Venta`: Tienen la única responsabilidad de proteger la integridad matemática de sus datos. `Inventario`: Administra la colección. No sabe de archivos ni de menús, solo gestiona objetos en memoria. `RepositorioCSV` y `Bitacora`: Son los únicos expertos en manipular el disco duro (`os` y `csv`). `SistemaInventario`: Es el director de orquesta. Es la única clase que se comunica con el usuario (`print` e `input`), capturando excepciones sin que la consola colapse.

3. Herencia y Polimorfismo: En la regla de negocio apliqué polimorfismo y herencia en el archivo `reglas.py` para cumplir con la regla seleccionada: Alerta de Stock Mínimo. Creé una interfaz base abstracta (`abc.ABC`) llamada `ReglaNegocio` que exige la implementación del método `revisar()`. De esta heredé la clase concreta `AlertaStockMinimo`. Decidí hacerlo aquí porque, si en el futuro la farmacia requiere nuevas reglas, el método de ventas en `app.py` no sufrirá modificaciones estructurales. Simplemente aplicará polimorfismo llamando a `regla.revisar(producto)` sin importarle qué regla específica se está ejecutando.

4. El mayor desafío de la refactorización: La parte más difícil fue el manejo seguro de los atributos. En el Proyecto 1, simplemente hacía `producto["stock"] = stock - cantidad`, lo cual permitía errores. Resolverlo en POO requirió dominar el uso de `@property` y `@setter`. Tuve que diseñar la clase `Producto` para que interceptara cualquier intento de modificación de los valores, lanzara una excepción personalizada (`StockInsuficienteError`) en caso de anomalías matemáticas, y delegar al `SistemaInventario` la tarea de capturar ese error y mostrarlo amigablemente.

5. Declaración de uso de herramientas: He utilizado la IA como apoyo metodológico para comprender a profundidad los pilares de la POO, la sintaxis correcta para las importaciones de módulos en Python y la generación de la estructura para los diagramas en lenguaje PlantUML, adaptando personalmente todo el código a mi lógica de farmacia y a mi regla de negocio de stock mínimo.